

Generating Physically Stable and Buildable Brick Structures from Text

Ava Pun* Kangle Deng* Ruixuan Liu* Deva Ramanan Changliu Liu Jun-Yan Zhu
 Carnegie Mellon University

Input Text Prompt: "A streamlined vessel with a long, narrow hull."

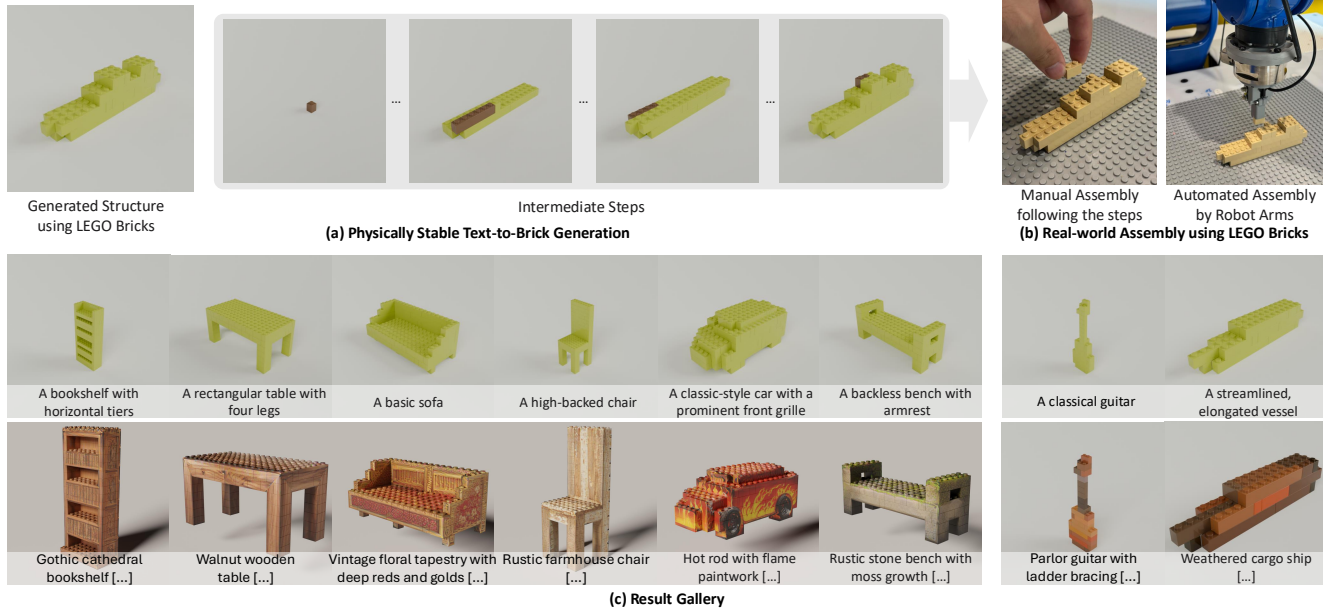


Figure 1. **Overview of BRICKGPT.** (a) Our method generates physically stable interconnecting brick assembly structures from text descriptions through an end-to-end approach, showing intermediate brick-by-brick steps. (b) The generated designs are buildable both by hand and by automated robots. (c) We show example results with corresponding text prompts. Besides basic brick designs (top), our method can generate colored (bottom right) and textured (bottom left) models with appearance descriptions. We highly recommend the reader to check our [website](https://avalovelace1.github.io/BrickGPT/) for step-by-step videos.

Abstract

We introduce BRICKGPT, the first approach for generating physically stable interconnecting brick assembly models from text prompts. To achieve this, we construct a large-scale, physically stable dataset of brick structures, along with their associated captions, and train an autoregressive large language model to predict the next brick to add via next-token prediction. To improve the stability of the resulting designs, we employ an efficient validity check and physics-aware rollback during autoregressive inference, which prunes infeasible token predictions using physics laws and assembly constraints. Our experiments show that BRICKGPT produces stable, diverse, and aesthetically pleasing brick structures that align closely with the input text prompts. We also develop a text-based brick texturing method to gen-

erate colored and textured designs. We show that our designs can be assembled manually by humans and automatically by robotic arms. We release our new dataset, Stable-Text2Brick, containing over 47,000 brick structures of over 28,000 unique 3D objects accompanied by detailed captions, along with our code and models at the project website: <https://avalovelace1.github.io/BrickGPT/>.

1. Introduction

3D generative models have made remarkable progress, driven by advances in generative modeling [16, 67] and neural rendering [27, 53]. These models have enabled various applications in virtual reality, gaming, entertainment, and scientific computing. Several works have explored synthesizing 3D objects from text [60], adding texture to meshes [8, 63], and manipulating the shape and appearance of existing 3D

*Indicates equal contribution.

objects and scenes [20, 41].

However, creating real-world objects with existing methods remains challenging. Most approaches focus on generating diverse 3D objects with high-fidelity geometry and appearance [22, 88], but these digital designs often cannot be physically realized due to two key challenges [47]. First, the objects may be difficult to assemble using standard components. Second, the resulting structure may be physically unstable even if assembly is possible. Without proper support, parts of the design could collapse, float, or be disconnected.

In this work, we address the challenge of generating *physically realizable objects*. We study this problem in the context of designing structures made of interlocking toy bricks, such as LEGO® blocks. These are widely used in education, artistic creation, and manufacturing prototyping. Additionally, they can serve as a reproducible benchmark, as all standard components are readily available. Due to the significant effort required to design brick structures manually, recent studies have developed automated algorithms to streamline the process and generate compelling results. However, existing approaches primarily create structures from a given 3D object [46] or focus on a single object category [13, 14].

Our goal is to generate brick assembly structures directly from freeform text prompts while ensuring physical stability and buildability. Specifically, we aim to train a generative model that produces designs that are:

- *Physically stable*: Built on a baseplate with strong structural integrity, without floating or collapsing bricks.
- *Buildable*: Compatible with standard toy brick pieces and able to be assembled brick-by-brick by humans or robots.

In this work, we introduce BRICKGPT with the key insight of repurposing autoregressive large language models, originally trained for next-token prediction, for next-brick prediction. We formulate the problem of brick structure design as an autoregressive text generation task, where the next-brick dimension and placement are specified with a simple textual format. To ensure generated structures are both *stable* and *buildable*, we enforce physics-aware assembly constraints during both training and inference. During training, we construct a large-scale dataset of physically stable brick structures paired with captions. During autoregressive inference, we enforce feasibility with an efficient validity check and physics-aware rollback to ensure that the final tokens adhere to physics laws and assembly constraints.

Our experiments show that the generated designs are stable, diverse, and visually appealing while adhering to input prompts. Our method outperforms pre-trained LLMs with or without in-context learning, and previous approaches based on 3D mesh generation. Finally, we explore applications such as text-driven brick texturing, as well as manual assembly and automated robotic assembly of our designs. Our dataset, code, and models are available at the project website: <https://avalovelace1.github.io/BrickGPT/>.

2. Related Work

Text-to-3D Generation. Text-to-3D generation has seen remarkable progress recently, driven by advances in neural rendering and generative models. Dreamfusion [60] and Score Jacobian Chaining [78] pioneer zero-shot text-to-3D generation by optimizing neural radiance fields [53] with pre-trained diffusion models [64]. Subsequent work has explored alternative 3D representations [3, 34, 36, 43, 49, 52, 68] and improved loss functions [26, 45, 48, 76, 80, 86]. Rather than relying on iterative optimization, a promising alternative direction trains generative models directly on 3D asset datasets, with various backbones including diffusion models [21, 33, 35, 55, 62, 65, 88, 89, 92], large reconstruction models [22, 32, 75, 84], U-Nets [37, 70], and autoregressive models [4–6, 19, 56, 66, 69, 82]. However, these existing methods cannot be directly applied to generating brick structures as they do not account for the unique physical and assembly constraints of real-world designs [47]. We bridge this gap by introducing a method for generating physically stable and buildable brick structures directly from text prompts.

Autoregressive 3D Modeling. Recent research has successfully used autoregressive models to generate 3D meshes [4–6, 9, 19, 56, 66, 69, 82], often conditioned on input text or images. Most recently, LLaMA-Mesh [81] demonstrates that large language models (LLMs) can be fine-tuned to output 3D shapes in plain-text format, given a text prompt. However, most existing autoregressive methods focus on mesh generation. In contrast, we focus on generating brick structures from text prompts, leveraging LLMs’ reasoning capabilities.

Brick Assembly and Design Generation. Creating brick structures given a reference 3D shape has been widely studied [29]. Existing works [58, 71, 91] formulate the generation as an optimization problem guided by hand-crafted heuristics, such as ensuring that all bricks are interconnected and minimizing the number of bricks. Wang et al. [79] translate a visual manual into step-by-step brick assembly instructions. Luo et al. [46] use stability analysis to find weak structural parts and rearrange the local brick layout to generate physically stable designs. Kim et al. [28], Liu et al. [40] formulate a planning problem to fill the target 3D model sequentially. However, these methods only generate designs given an input 3D shape, assuming a valid brick structure exists, which is difficult to verify in practice.

Few works have explored learning-based techniques to generate toy brick designs. Thompson et al. [72] use a deep graph generative model in which the graph encodes brick connectivity. However, this method is limited to generating simple classes using a single brick type. More recently, Ge et al. [14] use a diffusion model to predict a semantic volume, which is then translated into a high-quality micro building. Their method produces impressive results for a single category. Zhou et al. [90] and Ge et al. [13] generate compelling

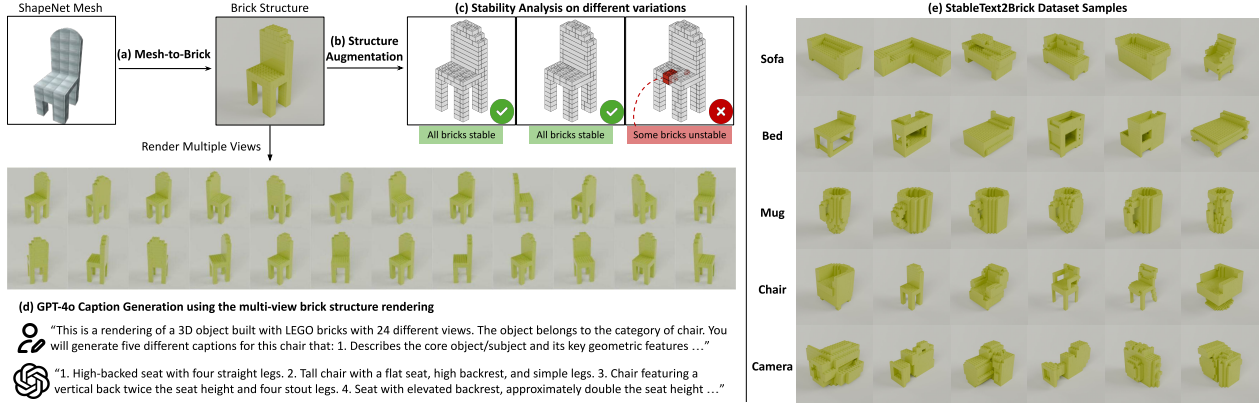


Figure 2. **StableText2Brick Dataset.** (a) From a ShapeNetCore [2] mesh, we generate a brick structure by voxelizing it onto a $20 \times 20 \times 20$ grid, then constructing its brick layout with a delete-and-rebuild algorithm. (b) We augment each shape with multiple structural variations by randomizing the brick layout while preserving the overall shape. (c) Stability analysis [38] is performed on each variation to filter out physically unstable designs. (d) To obtain captions for each shape, we render the brick structure from 24 different viewpoints and use GPT-4o [1] to generate detailed geometric descriptions. (e) Data samples from 5 categories in our StableText2Brick dataset.

figurine designs given an input portrait, selecting components from a pre-made set that best match an input photo. Although effective for faces, extending this selection-based approach to arbitrary objects is challenging. Zhou et al. [93] formulate an optimization problem to create a brick model from an input image. While their output is a 2D brick mosaic, we focus on 3D structures in this work. Goldberg et al. [15] query a vision-language model to generate diverse 3D assembly structures. However, they use regular building blocks instead of bricks with interlocking connections, and thus the structures have limited expressiveness. Lennon et al. [31] voxelize an input image and use heuristics to convert the voxels into a brick model without considering physical constraints. In contrast, our method performs the text-to-brick task without requiring intermediate voxel representations.

Physics-Aware Generation. Physics-aware 3D generation can be broadly categorized into two approaches: direct constraint enforcement and learned validation. Simple physical constraints, such as collision avoidance and contact requirements, can be incorporated directly through explicit penalty terms during optimization [17, 25, 42, 54, 77, 85, 87]. More complex physical properties, such as structural stability and dynamic behavior, typically require physics simulators [12, 15, 51, 57, 83] or data-driven physics-aware assessment models [10, 50]. To our knowledge, our paper is the first attempt to incorporate physics-aware constraints into text-based brick assembly structure generation.

3. Dataset

Training a modern autoregressive model requires a large-scale dataset. Therefore, we introduce StableText2Brick, a new dataset that contains 47,000+ different toy brick assembly structures, covering 28,000+ unique 3D objects from 21

common object categories of the ShapeNetCore dataset [2]. We select categories featuring diverse and distinctive 3D objects while excluding those resembling cuboids. Each structure is paired with a group of text descriptions and a stability score, which indicates its physical stability and buildability. Below, we describe the dataset construction, an overview of which is given in Figure 2.

Brick Representation. We consider brick structures built on a fixed baseplate. Each structure in StableText2Brick is represented as $B = [b_1, b_2, \dots, b_N]$ with N bricks, and each element denotes a brick’s state as $b_i = [h_i, w_i, x_i, y_i, z_i]$. Here, h_i and w_i indicate the brick length in the X and Y directions, respectively, and x_i , y_i , and z_i denote the position of the stud closest to the origin. The position has $x_i \in [0, 1, \dots, H - 1]$, $y_i \in [0, 1, \dots, W - 1]$, $z_i \in [0, 1, \dots, D - 1]$, where H , W , and D represent the dimensions of the discretized grid world.

Shape-to-Brick. We construct the dataset by converting 3D shapes from ShapeNetCore [2] into brick structures as shown in Figure 2(a). Given a 3D mesh, we voxelize and downsample it into a $20 \times 20 \times 20$ grid world to ensure a consistent scale, i.e., $H = W = D = 20$. The brick layout is generated by a delete-and-rebuild algorithm similar to [46]. To improve data quality and diversity, we introduce randomness and generate multiple different structures for the same 3D object, as illustrated in Figure 2(b). This increases the chance of obtaining a stable structure and more diverse layouts. More details can be found in Appendix A.

Stability Score. We assess the physical stability of each structure, as illustrated in Figure 2(c), using the analysis method [38]. For a structure $B = [b_1, b_2, \dots, b_N]$, the stability score $S \in \mathbb{R}^N$ assigns each brick b_i a value $s_i \in [0, 1]$ that quantifies the internal stress at its connections. Higher

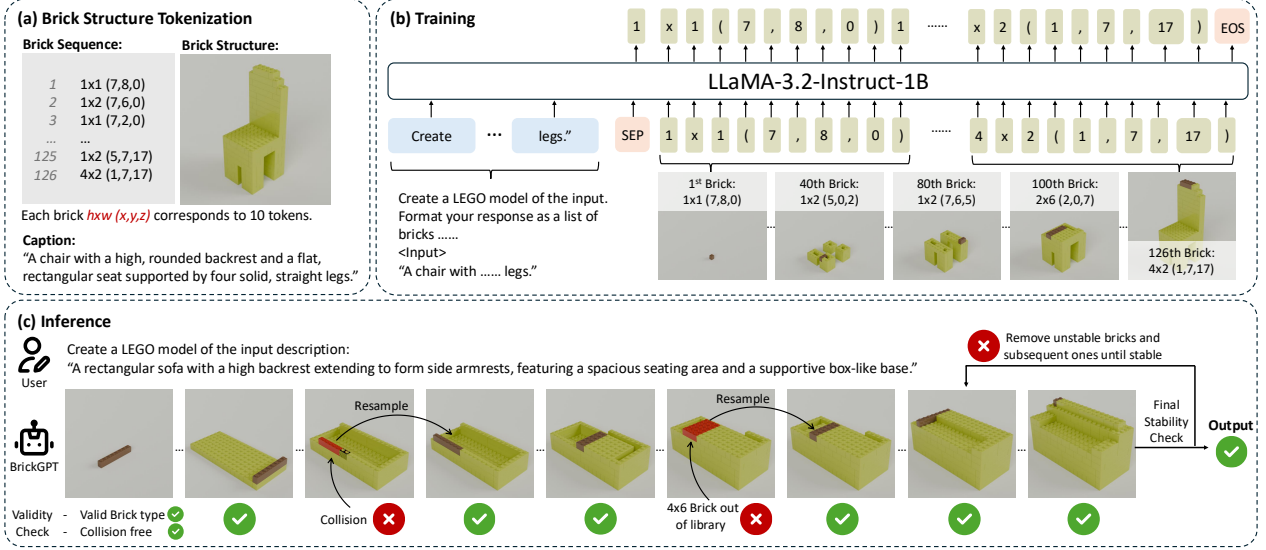


Figure 3. **Method.** (a) Our system tokenizes a brick structure into a sequence of text tokens, ordered in a raster-scan manner from bottom to top. (b) We create an instruction dataset pairing brick sequences with descriptions to fine-tune LLaMA-3.2-Instruct-1B. (c) At inference time, BRICKGPT generates brick structures incrementally by predicting one brick at a time given a text prompt. For each generated brick, we perform validity checks to ensure it is well-formatted, exists in our brick library, and does not collide with existing bricks. After completing the design, we verify its physical stability. If the structure is unstable, we roll back to a stable state by removing all unstable bricks and their subsequent bricks, and resume generation from that point.

scores s_i indicate greater stability, while $s_i = 0$ indicates an unstable brick that will cause structural failure. Calculating the stability score requires solving a nonlinear program to determine the forces acting on each brick to achieve a static equilibrium that prevents structural collapse, as detailed in Section 4.2. For typically-sized (i.e., < 200 bricks) structures in Figure 2, stability analysis takes ~ 0.35 seconds on average. A structure is stable if all bricks have stability scores greater than 0; we only include stable structures in the StableText2Brick dataset.

Caption Generation. To obtain captions for each shape, we render the brick structure from 24 different viewpoints and combine them into a single multi-view image. We then prompt GPT-4o [1] to produce five descriptions for these renderings with various levels of detail. Importantly, we ask GPT-4o to omit color information and focus only on geometry. The complete prompt is provided in Appendix A.

Figure 2(e) shows several data samples from StableText2Brick. The rich variations within each category and the comprehensive text-brick pairs make it possible to train large-scale generative models. More insights on StableText2Brick are discussed in Appendix A.

4. Method

Here, we introduce BRICKGPT, a method for generating physically stable interconnecting toy brick assembly structures from text prompts. Leveraging LLMs’ ability to model sequences and understand text, we fine-tune a pre-trained

LLM for the brick structure generation task (Section 4.1). To increase the stability and buildability of our designs, we use brick-by-brick rejection sampling and physics-aware roll-back during inference (Section 4.2). Figure 3 illustrates an overview of our method.

4.1. Model Fine-tuning

Pre-trained LLMs excel at modeling sequences and understanding natural language, making them promising candidates for our task. We further fine-tune a pre-trained LLM on a custom instruction dataset containing text prompts and their corresponding brick structures from StableText2Brick.

Pre-trained Base Model. We use LLaMA-3.2-1B-Instruct [11] as our base model. This model is fine-tuned to give coherent answers to instruction prompts, making it suitable for text-based brick structure generation. As shown in Figure 5, the base model can generate brick structures through in-context learning, highlighting the promise of using pre-trained LLMs for our task. However, the generated structures fail to follow the input prompt, and they contain intersecting or disconnected bricks, making them physically unstable and unbuildable. To address these issues, we further fine-tune the pre-trained model using our StableText2Brick.

Instruction Fine-tuning Dataset. For each stable structure and its corresponding captions, we construct an instruction in the following format: “(user) Create a LEGO model of {caption}. (assistant) {brick-structure}.” To simplify training and reuse LLaMA’s tokenizer, we represent brick structures

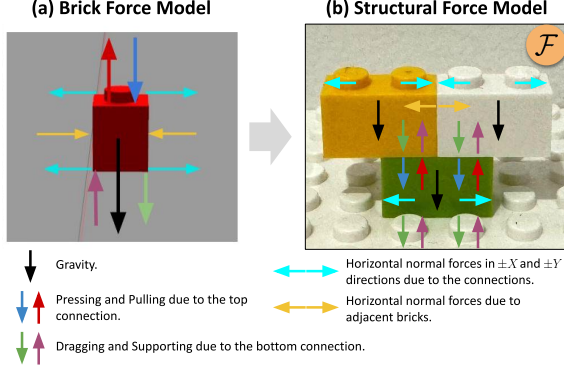


Figure 4. **Force Model.** (a) We consider all forces exerted on a single brick, including gravity (black), vertical forces with the top brick (red/blue) and bottom brick (green/purple), and horizontal (shear) forces due to knob connections (cyan), and adjacent bricks (yellow). (b) The structural force model $\mathcal{F}$ extends the individual force model to multiple bricks. Solving for static equilibrium in $\mathcal{F}$ determines each brick’s stability score.

in plain text. But what format should we use? The standard format LDraw [30] has two main drawbacks. First, it does not directly include brick dimensions, which are crucial for assessing the structure and validating brick placements. Second, it includes unnecessary information such as arbitrary brick orientation and scale, which are redundant since all bricks are axis-aligned in our setting.

Instead of using LDraw, we introduce a custom format to represent each brick structure. Each line of our format represents one brick as “ $\{h\} \times \{w\} (\{x\}, \{y\}, \{z\})$ ”, where $h \times w$ are brick dimensions and (x, y, z) are its coordinates. All bricks are 1-unit-tall, axis-aligned cuboids, and the order of h and w encodes the brick’s orientation about the vertical axis. This format significantly reduces the number of tokens required to represent a design, while including brick dimension information essential for 3D reasoning. Bricks are ordered in a raster-scan manner from bottom to top.

With our fine-tuned BRICKGPT model θ , we predict the bricks $b_1, b_2, \dots, b_N$ in an autoregressive manner:

$$p(b_1, b_2, \dots, b_N | \theta) = \prod_{i=1}^N p(b_i | b_1, \dots, b_{i-1}, \theta). \quad (1)$$

4.2. Integrating Physical Stability

Although trained on physically stable data, our model sometimes generates designs that violate physics and assembly constraints. To address this issue, we further incorporate physical stability verification into autoregressive inference.

A brick structure is considered physically stable and buildable if it does not collapse when built on a baseplate. To this end, we assess physical structural stability using the stability analysis method [38]. We briefly overview this method below. Figure 4(a) illustrates all possible forces exerted on

a single brick. We extend the single brick model and derive the structural force model $\mathcal{F}$, which consists of a set of candidate forces as shown in Figure 4(b). For a brick structure $B = [b_1, b_2, \dots, b_N]$, each brick b_i has M_i candidate forces $F_i^j \in \mathcal{F}_i, j \in [1, M_i]$. A structure is stable if all bricks can reach static equilibrium, i.e.,

$$\sum_j^{M_i} F_i^j = 0, \quad \sum_j^{M_i} \tau_i^j = \sum_j^{M_i} L_i^j \times F_i^j = 0, \quad (2)$$

where L_i^j denotes the force lever corresponding to F_i^j . The stability analysis is formulated into a nonlinear program as

$$\arg \min_{\mathcal{F}} \sum_i^N \left\{ \left| \sum_j^{M_i} F_i^j \right| + \left| \sum_j^{M_i} \tau_i^j \right| + \alpha \mathcal{D}_i^{\max} + \beta \sum \mathcal{D}_i \right\}, \quad (3)$$

subject to three constraints: 1) all force candidates in $\mathcal{F}$ should take non-negative values; 2) certain forces exerted on the same brick cannot coexist, e.g., the pulling (red arrow) and pressing (blue arrow), the dragging (green arrow) and supporting (purple arrow); 3) Newton’s third law, e.g., at a given connection point, the supporting force on the upper brick should be equal to the pressing force on the bottom brick. $\mathcal{D}_i \subset \mathcal{F}_i$ is the set of candidate dragging forces (green arrow) on b_i . α and β are hyperparameter weights.

Solving the above nonlinear program in Eqn. 3 using Gurobi [18] finds a force distribution $\mathcal{F}$ that drives the structure to static equilibrium with the minimum required internal stress, suppressing the overall friction (i.e., $\sum \mathcal{D}_i$) as well as avoiding extreme values (i.e., $\mathcal{D}_i^{\max}$). From the force distribution $\mathcal{F}$, we obtain the per-brick stability score as

$$s_i = \begin{cases} 0 & \begin{aligned} & \sum_j^{M_i} F_i^j \neq 0 \\ & \vee \sum_j^{M_i} \tau_i^j \neq 0 \\ & \vee \mathcal{D}_i^{\max} > F_T, \end{aligned} \\ \frac{F_T - \mathcal{D}_i^{\max}}{F_T} & \text{otherwise,} \end{cases} \quad (4)$$

where F_T is a measured constant friction capacity between brick connections. Higher scores s_i indicate greater stability, while $s_i = 0$ indicates an unstable brick that will cause structural failure: either $\mathcal{F}$ cannot reach static equilibrium ($\sum_j^{M_i} F_i^j \neq 0 \vee \sum_j^{M_i} \tau_i^j \neq 0$) or the required friction exceeds the friction capacity of the material ($\mathcal{D}_i^{\max} > F_T$). Due to the equality constraints imposed by Newton’s third law, Eqn. 3 includes only the dragging forces and excludes pulling forces. For a physically stable structure, we need $s_i > 0, \forall i \in [1, N]$.

When to apply stability analysis? A straightforward approach to ensuring physical stability is to apply stability analysis after each generated brick and resample a brick that would cause a collapse. However, this step-by-step validation could be time-consuming. More importantly, many

ALGORITHM 1: BRICKGPT inference algorithm.

Input: Text prompt c ; Autoregressive model θ
Output: Brick structure following the text prompt

```

1  $B \leftarrow$  empty brick structure;
2 loop
3   /* Predict next brick w/ rejection sampling */
4   for  $k = 1, \dots, \text{max\_rejections}$  do
5      $\text{context} \leftarrow T \oplus B.\text{to\_text\_format}()$ ;
6      $b \leftarrow \theta.\text{predict\_tokens}(\text{context})$  (Eqn. 1);
7     if  $b$  is valid then break;
8   end
9    $B.\text{add\_brick}(b)$ ;
10  if  $b$  contains EOF then // Structure complete
11    if  $B$  is stable or max rollbacks exceeded then return  $B$ ;
12    while  $B$  is unstable do // Rollback if unstable
13       $\mathcal{I} \leftarrow$  indices of unstable bricks in  $B$ ;
14       $i \leftarrow \min \mathcal{I}$ ; // idx of 1st unstable brick
15       $B \leftarrow [b_1, \dots, b_{i-1}]$ ;
16    end
17  end

```

structures are unstable during construction yet become stable when fully assembled; adding a stability check after each brick generation could overly constrain the model exploration space. Instead, we propose brick-by-brick rejection sampling combined with physics-aware rollback to balance stability and diversity.

Brick-by-Brick Rejection Sampling. To improve inference speed and avoid overly constraining generation, we relax our constraints during inference. First, when the model generates a brick and its position, the brick should be well-formatted (e.g., available in the inventory) and lie within the world space. Second, the brick should not collide with the existing structure. Formally, for each generated brick b_t , we have $\mathcal{V}_t \cap \mathcal{V}_i = \emptyset, \forall i \in [1, t-1]$, where $\mathcal{V}_i$ denotes the voxels occupied by b_i . These heuristics allow us to efficiently generate well-formatted structures without explicitly considering complex physical stability. To integrate these heuristics, we use rejection sampling: if a brick violates the rules, we resample a new brick from the model. Due to the relaxed constraints, most bricks are valid, and rejection sampling does not significantly affect inference time.

Physics-Aware Rollback. To ensure that the final design $B = [b_1, b_2, \dots, b_N]$ is physically stable, we calculate the stability score S . If the resulting design is unstable, i.e., $s_i = 0, i \in \mathcal{I}$, we roll back the design to the state before the first unstable brick was generated, i.e., $B' = [b_1, b_2, \dots, b_{\min \mathcal{I}-1}]$. Here, $\mathcal{I}$ is the set of the indices of all the unstable bricks. We repeat this process iteratively until we reach a stable structure B' , and continue generation from the partial structure B' . Note that we can use the per-brick stability score to efficiently find the collapsing bricks and their corresponding indices in the sequence. We summarize our inference sampling in Algorithm 1.

4.3. Brick Texturing and Coloring

While we primarily focus on generating the *shape* of a brick structure, color and texture play a critical role in creative designs. Therefore, we propose a method that applies detailed UV textures or assigns uniform colors to individual bricks.

UV Texture Generation. Given a structure B and its corresponding mesh $\mathcal{M}$, we first identify the set of occluded bricks B_{occ} that have all six faces covered by adjacent bricks, and remove B_{occ} for efficiency. The remaining bricks $B_{\text{vis}} = B \setminus B_{\text{occ}}$ are merged into a single mesh $\mathcal{M}$ with cleaned overlapping vertices using *ImportLDraw* [74]. We generate a UV map $\text{UV}_{\mathcal{M}}$ by cube projection. The texture map I_{texture} is then generated using FlashTex [8], a fast text-based mesh texturing approach:

$$I_{\text{texture}} = \text{FlashTex}(\mathcal{M}, \text{UV}_{\mathcal{M}}, c), \quad (5)$$

where text prompt c describes the visual appearance. This texture can be applied through UV printing or stickers.

Uniform Brick Color Assignment. We can also assign each brick a uniform color from a standard color library [30]. Given a structure B , we convert it to a voxel grid $\mathcal{V}$ and then to a UV-unwrapped mesh $\mathcal{M}_{\mathcal{V}}$. For every voxel $v \in \mathcal{V}$, let $f_i^v, i = 1, \dots, N_v$ be its visible faces where $0 \leq N_v \leq 6$. Each face f_i^v is split into two triangles and mapped to a UV region $\mathcal{S}_i^v$, creating a mesh $\mathcal{M}_{\mathcal{V}}$ with UV map $\text{UV}_{\mathcal{V}}$. We apply FlashTex [8] to generate a texture I_{texture} :

$$I_{\text{texture}} = \text{FlashTex}(\mathcal{M}_{\mathcal{V}}, \text{UV}_{\mathcal{V}}, c). \quad (6)$$

The color of each voxel $\mathcal{C}(v) \in \mathbb{R}^3$ is computed as:

$$\mathcal{C}(v) = \frac{1}{N_v} \sum_{i=1}^{N_v} \mathcal{C}(f_i^v), \quad \forall v \in \mathcal{V}, \quad (7)$$

where $\mathcal{C}(f_i^v) = \frac{1}{|\mathcal{S}_i^v|} \sum_{(x,y) \in \mathcal{S}_i^v} I_{\text{texture}}(x, y)$ is the color of each visible face f_i^v , and $|\mathcal{S}_i^v|$ represents the number of pixels in region $\mathcal{S}_i^v$ in the UV map. For each brick b_t and its constituent voxels $\mathcal{V}_t$, we compute the brick color $\mathcal{C}(b_t) = \frac{1}{|\mathcal{V}_t|} \sum_{v \in \mathcal{V}_t} \mathcal{C}(v)$. Finally, we find the closest color in the color set. While UV texturing offers higher-fidelity details, uniform coloring allows us to use standard toy bricks.

5. Experiments

5.1. Implementation Details

Fine-tuning. Our fine-tuning dataset contains 240k distinct prompts and 47k+ distinct brick structures. We withhold 10% of the data for evaluation. For efficiency, we include samples only up to 4096 tokens long. Training details are provided in Appendix A.

Inference. To evaluate our method, we generate one brick structure for each of 250 prompts randomly selected from

Table 1. **Quantitative Results.** We evaluate our method against several baselines on validity (no out-of-library, out-of-bounds, or colliding bricks), stability, CLIP-based text similarity, and DINOv2-based image similarity. Stability, CLIP, and DINO are computed over valid structures only. For LLaMA-Mesh [81], validity requires a well-formed OBJ file. Results marked “+ our stability analysis” are augmented by generating multiple structures and choosing the first stable one found (if any). Our method outperforms all baselines as well as the ablated setups on validity and stability using our proposed rejection sampling and rollback, while maintaining high text similarity.

Method	% valid	% stable	mean stability	min stability	CLIP	DINO
Pre-trained LLaMA (0-shot)	0.0%	0.0%	N/A	N/A	N/A	N/A
In-context learning (5-shot)	2.4%	1.2%	0.675	0.479	0.284	0.814
LLaMA-Mesh [81]	94.8%	50.8%	0.894	0.499	0.317	0.851
LGM [70]	100%	25.2%	0.942	0.231	0.300	0.851
XCube [62]	100%	75.2%	0.964	0.686	0.322	0.859
Hunyuan3D-2 [89]	100%	75.2%	0.973	0.704	<u>0.324</u>	0.868
LLaMA-Mesh [81] + our stability analysis	94.8%	58.0%	0.896	0.564	0.317	0.851
LGM [70] + our stability analysis	100%	32.5%	0.941	0.285	0.300	0.851
XCube [62] + our stability analysis	100%	83.6%	0.963	0.754	0.322	0.859
Hunyuan3D-2 [89] + our stability analysis	100%	88.4%	0.976	0.813	<u>0.324</u>	0.868
Ours w/o rejection sampling or rollback	37.2%	12.8%	0.956	0.325	0.329	0.888
Ours w/o rollback	100%	24.0%	0.947	0.228	0.322	<u>0.882</u>
Ours (BRICKGPT)	100%	98.8%	0.996	0.915	<u>0.324</u>	0.880

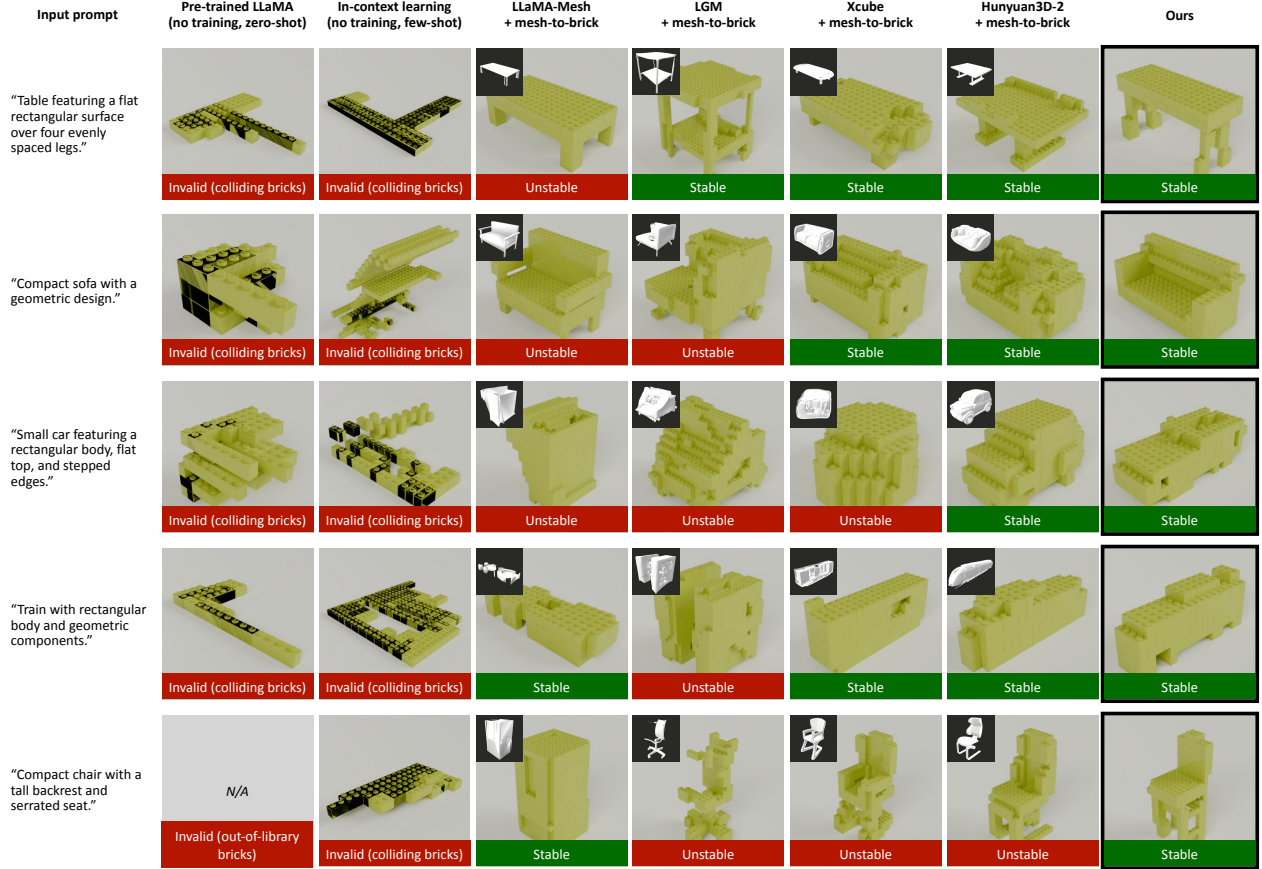


Figure 5. **Result Gallery and Baseline Comparisons.** Our method generates high-quality, diverse, and novel brick structures aligned with the given text prompts. Black bricks are colliding. For LLaMA-Mesh [81], LGM [70], XCube [62], and Hunyuan3D-2 [89], an inset of the generated mesh is shown in the top-left corner.

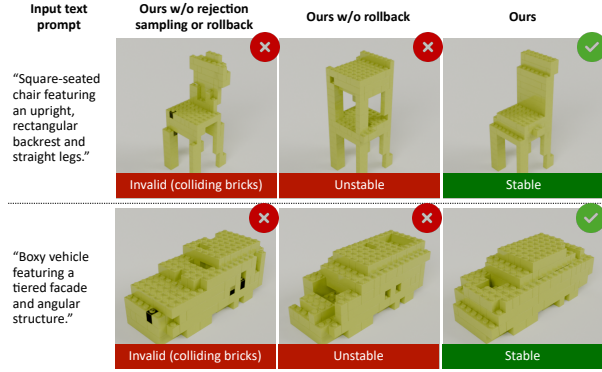


Figure 6. **Ablation Study.** Brick-by-brick rejection sampling and physics-informed rollback help to ensure that the generated structure is both valid and stable. Black indicates colliding bricks.

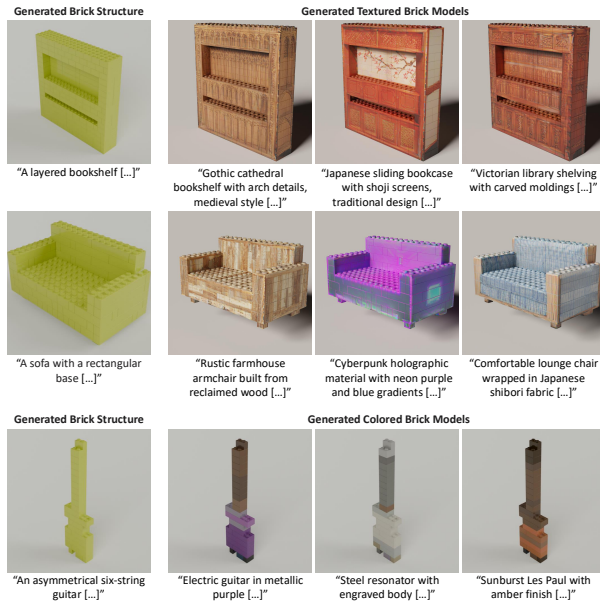


Figure 7. **Brick Texture and Color Generation.** Our method can generate diverse textured (top two rows) and colored (bottom) structures based on the same shape with different appearance prompts.

the validation dataset. The nonlinear optimization in Eqn. 3 is solved using Gurobi [18]. We set $F_T = 0.98N$ with $\alpha = 10^{-3}$ and $\beta = 10^{-6}$. We allow up to 100 physics-aware rollbacks before accepting the brick structure. The median number of required rollbacks is 2, and the median time to generate one structure is 40.8 seconds.

5.2. Brick Structure Generation Results

Figure 5 shows a gallery of diverse, high-quality brick structures that closely follow the input prompts.

Baseline Comparisons. As baselines, we use LLaMA-Mesh [81], LGM [70], XCube [62], and Hunyuan3D-2 [89] to generate a mesh from each prompt, then convert the

meshes to brick structures with our delete-and-rebuild algorithm. To increase the chance of producing a stable structure, we generate multiple different structures for the same output mesh (10 for LLaMA-Mesh and LGM; 100 for XCube and Hunyuan3D) and accept the first stable one found, if any. Additionally, we compare our method with pre-trained LLaMA models evaluated in both a zero-shot and few-shot manner. For few-shot evaluation, we provide the model with 5 examples of stable brick structures and their captions.

For each method, we compute the proportion of stable and valid structures among the generated designs. Additionally, for each valid structure, we compute its mean and minimum brick stability scores. To evaluate prompt alignment, we compute the CLIP score [61] between a rendered image of each valid structure and the text “A LEGO model of {prompt}”. We also calculate the alignment between rendered images of the generated structure and the ground-truth structure for the same prompt, as measured by the cosine similarity between DINOv2 features [59]. As shown in Table 1, our method outperforms all baselines in these metrics.

Ablation Study. We demonstrate the importance of rejection sampling and physics-aware rollback. As seen in Figure 6, rejection sampling eliminates invalid bricks, such as those with collisions, while rollback helps to ensure the final assembly structure is stable. The quantitative results in Table 1 show that our full method generates a higher proportion of valid and stable brick structures, while closely following the text prompts.

5.3. Extensions and Applications

Robotic Assembly of Generated Structures. We demonstrate automated assembly using a dual-robot-arm system. The robots use the manipulation policy [39] and the asynchronous multi-agent planner [24] to manipulate toy bricks and construct the structure. More details are included in Appendix B. Since the generated structures are physically stable, efficient automated assembly can be performed.

Manual Assembly. Our generated designs are physically valid and can be hand-assembled. See Appendix C.

Brick Texture and Color Generation. Figure 7 shows both UV texturing and uniform coloring results of brick structures, demonstrating our method’s ability to generate diverse styles while preserving the underlying geometry.

6. Conclusion

In this work, we have introduced BRICKGPT, an autoregressive model for generating interconnecting toy brick structures from text prompts. Our method predicts the next brick sequentially while ensuring physical stability and buildability. We have shown that our method outperforms LLM backbone models and recent text-to-3D generation methods.

Acknowledgments

We thank Minchen Li, Ken Goldberg, Nupur Kumari, Ruihan Gao, and Yihao Shi for their discussions and help. We also thank Jiaoyang Li, Philip Huang, and Shobhit Aggarwal for developing the bimanual robotic system. This work is partly supported by the Packard Foundation, Cisco Research Grant, and Amazon Faculty Award. This work is also in part supported by the Manufacturing Futures Institute, Carnegie Mellon University, through a grant from the Richard King Mellon Foundation. KD is supported by the Microsoft Research PhD Fellowship.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 3, 4
- [2] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An Information-Rich 3D Model Repository. Technical Report arXiv:1512.03012 [cs.GR], Stanford University — Princeton University — Toyota Technological Institute at Chicago, 2015. 3, 13
- [3] Rui Chen, Yongwei Chen, Ningxin Jiao, and Kui Jia. Fantasia3D: Disentangling geometry and appearance for high-quality text-to-3D content creation. In *IEEE International Conference on Computer Vision (ICCV)*, 2023. 2
- [4] Sijin Chen, Xin Chen, Anqi Pang, Xianfang Zeng, Wei Cheng, Yijun Fu, Fukun Yin, Yanru Wang, Zhibin Wang, Chi Zhang, Jingyi Yu, Gang Yu, Bin Fu, and Tao Chen. MeshXL: Neural coordinate field for generative 3D foundation models. *arXiv preprint arXiv:2405.20853*, 2024. 2
- [5] Yiwen Chen, Tong He, Di Huang, Weicai Ye, Sijin Chen, Jiaxiang Tang, Xin Chen, Zhongang Cai, Lei Yang, Gang Yu, Guosheng Lin, and Chi Zhang. MeshAnything: Artist-created mesh generation with autoregressive transformers. *arXiv preprint arXiv:2406.10163*, 2024.
- [6] Yiwen Chen, Yikai Wang, Yihao Luo, Zhengyi Wang, Zilong Chen, Jun Zhu, Chi Zhang, and Guosheng Lin. MeshAnything V2: Artist-created mesh generation with adjacent mesh tokenization. *arXiv preprint arXiv:2408.02555*, 2024. 2
- [7] Matt Deitke, Ruoshi Liu, Matthew Wallingford, Huong Ngo, Oscar Michel, Aditya Kusupati, Alan Fan, Christian Laforte, Vikram Voleti, Samir Yitzhak Gadre, Eli VanderBilt, Anirudha Kembhavi, Carl Vondrick, Georgia Gkioxari, Kiana Ehsani, Ludwig Schmidt, and Ali Farhadi. Objaverse-XL: A universe of 10M+ 3D objects. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 35799–35813. Curran Associates, Inc., 2023. 14
- [8] Kangle Deng, Timothy Omernick, Alexander Weiss, Deva Ramanan, Jun-Yan Zhu, Tinghui Zhou, and Maneesh Agrawala. FlashTex: Fast relightable mesh texturing with LightControlNet. In *European Conference on Computer Vision (ECCV)*, 2024. 1, 6
- [9] Kangle Deng, Hsueh-Ti Derek Liu, Yiheng Zhu, Xiaoxia Sun, Chong Shang, Kiran Bhat, Deva Ramanan, Jun-Yan Zhu, Maneesh Agrawala, and Tinghui Zhou. Efficient autoregressive shape generation via octree-based adaptive tokenization. *arXiv preprint arXiv:2504.02817*, 2025. 2
- [10] Yuan Dong, Qi Zuo, Xiaodong Gu, Weihao Yuan, Zhengyi Zhao, Zilong Dong, Liefeng Bo, and Qixing Huang. GPLD3D: Latent diffusion of 3D shape generative models by enforcing geometric and physical priors. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 3
- [11] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 4, 14
- [12] Pablo Funes and Jordan Pollack. Evolutionary body building: Adaptive physical designs for robots. *Artificial Life*, 4(4): 337–357, 1998. 3
- [13] Jiahao Ge, Mingjun Zhou, Wenrui Bao, Hao Xu, and Chi-Wing Fu. Creating LEGO figurines from single images. *ACM Transactions on Graphics (TOG)*, 43(4):153:1–153:16, 2024. 2
- [14] Jiahao Ge, Mingjun Zhou, and Chi-Wing Fu. Learn to create simple LEGO micro buildings. *ACM Transactions on Graphics (TOG)*, 43(6):249:1–249:13, 2024. 2
- [15] Andrew Goldberg, Kavish Kondap, Tianshuang Qiu, Zehan Ma, Letian Fu, Justin Kerr, Huang Huang, Kaiyuan Chen, Kuan Fang, and Ken Goldberg. Blox-Net: Generative design-for-robot-assembly using VLM supervision, physics, simulation, and a robot with reset. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 2025. 3
- [16] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. 1
- [17] Minghao Guo, Bohan Wang, Pingchuan Ma, Tianyuan Zhang, Crystal Elaine Owens, Chuang Gan, Joshua B. Tenenbaum, Kaiming He, and Wojciech Matusik. Physically compatible 3D object modeling from a single image. *arXiv preprint arXiv:2405.20510*, 2024. 3
- [18] Gurobi Optimization, LLC. Gurobi Optimizer reference manual, 2023. 5, 8
- [19] Zekun Hao, David W Romero, Tsung-Yi Lin, and Ming-Yu Liu. Meshton: High-fidelity, artist-like 3D mesh generation at scale. *arXiv preprint arXiv:2412.09548*, 2024. 2
- [20] Ayaan Haque, Matthew Tancik, Alexei Efros, Aleksander Holynski, and Angjoo Kanazawa. Instruct-NeRF2NeRF: Editing 3D scenes with instructions. In *IEEE International Conference on Computer Vision (ICCV)*, 2023. 2
- [21] Fangzhou Hong, Jiaxiang Tang, Ziang Cao, Min Shi, Tong Wu, Zhaoxi Chen, Tengfei Wang, Liang Pan, Dahua Lin, and Ziwei Liu. 3DTopia: Large text-to-3D generation model with hybrid diffusion priors. *arXiv preprint arXiv:2403.02234*, 2024. 2
- [22] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao

- Tan. LRM: Large reconstruction model for single image to 3D. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [23] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 14
- [24] Philip Huang, Ruixuan Liu, Shobhit Aggarwal, Changliu Liu, and Jiaoyang Li. Apex-mr: Multi-robot asynchronous planning and execution for cooperative assembly. In *Robotics: Science and Systems*, 2025. 8, 14
- [25] Siyuan Huang, Zan Wang, Puhao Li, Baoxiong Jia, Tengyu Liu, Yixin Zhu, Wei Liang, and Song-Chun Zhu. Diffusion-based generation, optimization, and planning in 3D scenes. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 3
- [26] Oren Katzir, Or Patashnik, Daniel Cohen-Or, and Dani Lischinski. Noise-free score distillation. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [27] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. In *ACM Transactions on Graphics (TOG)*, 2023. 1
- [28] Jungtaek Kim, Hyunsoo Chung, Jinhwi Lee, Minsu Cho, and Jaesik Park. Combinatorial 3D shape generation via sequential assembly. *arXiv preprint arXiv:2004.07414*, 2020. 2
- [29] Jae Woo Kim. Survey on automated LEGO assembly construction. 2014. 2
- [30] LDraw.org. Ldraw.org homepage, 2025. 5, 6
- [31] Kyle Lennon, Katharina Fransen, Alexander O’Brien, Yumeng Cao, Matthew Beveridge, Yamin Arefeen, Nikhil Singh, and Iddo Drori. Image2Lego: Customized LEGO set generation from images. *arXiv preprint arXiv:2108.08477*, 2021. 3
- [32] Jiahao Li, Hao Tan, Kai Zhang, Zexiang Xu, Fujun Luan, Yinghao Xu, Yicong Hong, Kalyan Sunkavalli, Greg Shakhnarovich, and Sai Bi. Instant3D: Fast text-to-3D with sparse-view generation and large reconstruction model. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [33] Muheng Li, Yueqi Duan, Jie Zhou, and Jiwen Lu. Diffusion-SDF: Text-to-shape via voxelized diffusion. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [34] Weiyu Li, Rui Chen, Xuelin Chen, and Ping Tan. Sweet-Dreamer: Aligning geometric priors in 2D diffusion for consistent text-to-3D. *arXiv preprint arXiv:2310.02596*, 2023. 2
- [35] Weiyu Li, Jiarui Liu, Rui Chen, Yixun Liang, Xuelin Chen, Ping Tan, and Xiaoxiao Long. CraftsMan: High-fidelity mesh generation with 3D native generation and interactive geometry refiner. *arXiv preprint arXiv:2405.14979*, 2024. 2
- [36] Chen-Hsuan Lin, Jun Gao, Luming Tang, Towaki Takikawa, Xiaohui Zeng, Xun Huang, Karsten Kreis, Sanja Fidler, Ming-Yu Liu, and Tsung-Yi Lin. Magic3D: High-resolution text-to-3D content creation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [37] Minghua Liu, Chong Zeng, Xinyue Wei, Ruoxi Shi, Linghao Chen, Chao Xu, Mengqi Zhang, Zhaoning Wang, Xiaoshuai Zhang, Isabella Liu, et al. MeshFormer: High-quality mesh generation with 3D-guided reconstruction model. *Advances in Neural Information Processing Systems (NeurIPS)*, 37: 59314–59341, 2025. 2
- [38] Ruixuan Liu, Kangle Deng, Ziwei Wang, and Changliu Liu. StableLego: Stability analysis of block stacking assembly. *IEEE Robotics and Automation Letters*, 9(11):9383–9390, 2024. 3, 5, 13
- [39] Ruixuan Liu, Yifan Sun, and Changliu Liu. A lightweight and transferable design for robust LEGO manipulation. *International Symposium on Flexible Automation*, page V001T07A004, 2024. 8, 14
- [40] Ruixuan Liu, Alan Chen, Weiye Zhao, and Changliu Liu. Physics-aware combinatorial assembly sequence planning using data-free action masking. *IEEE Robotics and Automation Letters*, 10(5):4882–4889, 2025. 2, 14
- [41] Steven Liu, Xiuming Zhang, Zhoutong Zhang, Richard Zhang, Jun-Yan Zhu, and Bryan Russell. Editing conditional radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, pages 5773–5783, 2021. 2
- [42] Xueyi Liu, Bin Wang, He Wang, and Li Yi. Few-shot physically-aware articulated mesh generation via hierarchical deformation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 3
- [43] Xiaoxiao Long, Yuan-Chen Guo, Cheng Lin, Yuan Liu, Zhiyang Dou, Lingjie Liu, Yuexin Ma, Song-Hai Zhang, Marc Habermann, Christian Theobalt, et al. Wonder3D: Single image to 3D using cross-domain diffusion. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
- [44] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 14
- [45] Artem Lukoianov, Haitz Sáez de Ocáriz Borde, Kristjan Greenwald, Vitor Campagnolo Guizilini, Timur Bagautdinov, Vincent Sitzmann, and Justin Solomon. Score distillation via reparametrized DDIM. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2
- [46] Sheng-Jie Luo, Yonghao Yue, Chun-Kai Huang, Yu-Huan Chung, Sei Imai, Tomoyuki Nishita, and Bing-Yu Chen. Legolization: optimizing LEGO designs. *ACM Transactions on Graphics (TOG)*, 34(6), 2015. 2, 3, 13
- [47] Liane Makatura, Michael Foshey, Bohan Wang, Felix Hahn-Lein, Pingchuan Ma, Bolei Deng, Megan Tjandrasuwita, Andrew Spielberg, Crystal Elaine Owens, Peter Yichen Chen, et al. How can large language models help humans in design and manufacturing? *arXiv preprint arXiv:2307.14377*, 2023. 2
- [48] David McAllister, Songwei Ge, Jia-Bin Huang, David W. Jacobs, Alexei A. Efros, Aleksander Holynski, and Angjoo Kanazawa. Rethinking score distillation as a bridge between image distributions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2
- [49] Gal Metzer, Elad Richardson, Or Patashnik, Raja Giryes, and Daniel Cohen-Or. Latent-NeRF for shape-guided generation

- of 3d shapes and textures. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [50] Mariem Mezghanni, Malika Boulkenafed, Andre Lieutier, and Maks Ovsjanikov. Physically-aware generative network for 3D shape modeling. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 3
- [51] Mariem Mezghanni, Théo Bodrito, Malika Boulkenafed, and Maks Ovsjanikov. Physical simulation layer for accurate 3D modeling. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 3
- [52] Oscar Michel, Roi Bar-On, Richard Liu, Sagie Benaim, and Rana Hanocka. Text2Mesh: Text-driven neural stylization for meshes. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2
- [53] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020. 1, 2
- [54] Vihaan Misra, Peter Schaldenbrand, and Jean Oh. ShapeShift: Towards text-to-shape arrangement synthesis with content-aware geometric constraints. *arXiv preprint arXiv:2503.14720*, 2025. 3
- [55] Gimin Nam, Mariem Khelifi, Andrew Rodriguez, Alberto Tono, Linqi Zhou, and Paul Guerrero. 3D-LDM: Neural implicit 3D shape generation with latent diffusion models. *arXiv preprint arXiv:2212.00842*, 2022. 2
- [56] Charlie Nash, Yaroslav Ganin, S. M. Ali Eslami, and Peter Battaglia. PolyGen: An autoregressive generative model of 3D meshes. In *International Conference on Machine Learning (ICML)*, pages 7220–7229. PMLR, 2020. 2
- [57] Junfeng Ni, Yixin Chen, Bohan Jing, Nan Jiang, Bin Wang, Bo Dai, Puhao Li, Yixin Zhu, Song-Chun Zhu, and Siyuan Huang. PhyRecon: Physically plausible neural scene reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 3
- [58] Sumiaki Ono, Alexis Andre, Youngha Chang, and Masayuki Nakajima. LEGO builder: Automatic generation of LEGO assembly manual from 3D polygon model. *ITE Transactions on Media Technology and Applications*, 1:354–360, 2013. 2
- [59] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel HAZIZA, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. Featured Certification. 8
- [60] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. DreamFusion: Text-to-3D using 2D diffusion. In *International Conference on Learning Representations (ICLR)*, 2023. 1, 2
- [61] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021. 8
- [62] Xuanchi Ren, Jiahui Huang, Xiaohui Zeng, Ken Museth, Sanja Fidler, and Francis Williams. XCube: Large-scale 3D generative modeling using sparse voxel hierarchies. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4209–4219, 2024. 2, 7, 8
- [63] Elad Richardson, Gal Metzer, Yuval Alaluf, Raja Giryes, and Daniel Cohen-Or. Texture: Text-guided texturing of 3D shapes. In *ACM SIGGRAPH*, 2023. 1
- [64] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2
- [65] J Ryan Shue, Eric Ryan Chan, Ryan Po, Zachary Ankner, Jiajun Wu, and Gordon Wetzstein. 3D neural field generation using triplane diffusion. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [66] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. MeshGPT: Generating triangle meshes with decoder-only transformers. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19615–19625, 2024. 2
- [67] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning (ICML)*, 2015. 1
- [68] Jingxiang Sun, Bo Zhang, Ruizhi Shao, Lizhen Wang, Wen Liu, Zhenda Xie, and Yebin Liu. DreamCraft3D: Hierarchical 3D generation with bootstrapped diffusion prior. *arXiv preprint arXiv:2310.16818*, 2023. 2
- [69] Jiaxiang Tang, Zhaoshuo Li, Zekun Hao, Xian Liu, Gang Zeng, Ming-Yu Liu, and Qinsheng Zhang. EdgeRunner: Autoregressive auto-encoder for artistic mesh generation. *arXiv preprint arXiv:2409.18114*, 2024. 2
- [70] Jiaxiang Tang, Zhaoxi Chen, Xiaokang Chen, Tengfei Wang, Gang Zeng, and Ziwei Liu. LGM: Large multi-view gaussian model for high-resolution 3D content creation. In *European Conference on Computer Vision (ECCV)*, pages 1–18, Cham, 2025. Springer Nature Switzerland. 2, 7, 8
- [71] Romain Pierre Testuz, Yuliy Schwartzburg, and Mark Pauly. Automatic generation of constructable brick sculptures. Technical report, École Polytechnique Fédérale de Lausanne, 2013. 2
- [72] Rylee Thompson, Elahe Ghalebi, Terrance DeVries, and Graham W. Taylor. Building LEGO using deep generative models of graphs. *arXiv preprint arXiv:2012.11543*, 2020. 2
- [73] Yunsheng Tian, Jie Xu, Yichen Li, Jieliang Luo, Shinjiro Sueda, Hui Li, Karl D.D. Willis, and Wojciech Matusik. Assemble them all: Physics-based planning for generalizable assembly by disassembly. *ACM Transactions on Graphics (TOG)*, 41(6), 2022. 14
- [74] TobyLobster. ImportLDraw - blender add-on for importing LDraw models. <https://github.com/>

[TobyLobster/ImportLDraw](#), 2025. Accessed: 2025-03-07. 6

- [75] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. TripoSR: Fast 3D object reconstruction from a single image. *arXiv preprint arXiv:2403.02151*, 2024. 2
- [76] Uy Dieu Tran, Minh Luu, Phong Ha Nguyen, Khoi Nguyen, and Binh-Son Hua. Diverse text-to-3D synthesis with augmented text embedding. In *European Conference on Computer Vision (ECCV)*, 2024. 2
- [77] Alexander Vilesov, Pradyumna Chari, and Achuta Kadambi. CG3D: Compositional generation for text-to-3D via Gaussian splatting. *arXiv preprint arXiv:2311.17907*, 2023. 3
- [78] Haochen Wang, Xiaodan Du, Jiahao Li, Raymond A. Yeh, and Greg Shakhnarovich. Score Jacobian chaining: Lifting pre-trained 2D diffusion models for 3D generation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
- [79] Ruocheng Wang, Yunzhi Zhang, Jiayuan Mao, Chin-Yi Cheng, and Jiajun Wu. Translating a visual lego manual to a machine-executable plan. In *European Conference on Computer Vision (ECCV)*, 2022. 2
- [80] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. ProlificDreamer: High-fidelity and diverse text-to-3D generation with variational score distillation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2
- [81] Zhengyi Wang, Jonathan Lorraine, Yikai Wang, Hang Su, Jun Zhu, Sanja Fidler, and Xiaohui Zeng. LLaMA-Mesh: Unifying 3D mesh generation with language models. *arXiv preprint arXiv:2411.09595*, 2024. 2, 7, 8
- [82] Haohan Weng, Zibo Zhao, Biwen Lei, Xianghui Yang, Jian Liu, Zeqiang Lai, Zhuo Chen, Yuhong Liu, Jie Jiang, Chunchao Guo, et al. Scaling mesh generation via compressive tokenization. *arXiv preprint arXiv:2411.07025*, 2024. 2
- [83] Qingshan Xu, Jiao Liu, Melvin Wong, Caishun Chen, and Yew-Soon Ong. Precise-physics driven text-to-3D generation. *arXiv preprint arXiv:2403.12438*, 2024. 3
- [84] Yinghao Xu, Hao Tan, Fujun Luan, Sai Bi, Peng Wang, Jiahao Li, Zifan Shi, Kalyan Sunkavalli, Gordon Wetzstein, Zexiang Xu, and Kai Zhang. DMV3D: Denoising multi-view diffusion using 3D large reconstruction model. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [85] Yandan Yang, Baoxiong Jia, Peiyuan Zhi, and Siyuan Huang. PhyScene: Physically interactable 3D scene synthesis for embodied AI. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 3
- [86] Junliang Ye, Fangfu Liu, Qixiu Li, Zhengyi Wang, Yikai Wang, Xinzhou Wang, Yueqi Duan, and Jun Zhu. Dream-Reward: Text-to-3D generation with human preference. In *European Conference on Computer Vision (ECCV)*, 2024. 2
- [87] Ye Yuan, Jiaming Song, Umar Iqbal, Arash Vahdat, and Jan Kautz. PhysDiff: Physics-guided human motion diffusion model. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 3
- [88] Longwen Zhang, Ziyu Wang, Qixuan Zhang, Qiwei Qiu, Anqi Pang, Haoran Jiang, Wei Yang, Lan Xu, and Jingyi Yu. CLAY: A controllable large-scale generative model for creating high-quality 3D assets. In *ACM Transactions on Graphics (TOG)*, 2024. 2
- [89] Zibo Zhao, Zeqiang Lai, Qingxiang Lin, Yunfei Zhao, Haolin Liu, Shuhui Yang, Yifei Feng, Mingxin Yang, Sheng Zhang, Xianghui Yang, et al. Hunyuan3D 2.0: Scaling diffusion models for high resolution textured 3D assets generation. *arXiv preprint arXiv:2501.12202*, 2025. 2, 7, 8
- [90] Guyue Zhou, Liyi Luo, Hao Xu, Xinliang Zhang, Haole Guo, and Hao Zhao. Brick yourself within 3 minutes. In *International Conference on Robotics and Automation (ICRA)*, pages 6261–6267, 2022. 2
- [91] J. Zhou, X. Chen, and Y. Xu. Automatic generation of vivid LEGO architectural sculptures. *Computer Graphics Forum*, 2019. 2
- [92] Linqi Zhou, Yilun Du, and Jiajun Wu. 3D shape generation and completion through point-voxel diffusion. In *IEEE International Conference on Computer Vision (ICCV)*, 2021. 2
- [93] Mingjun Zhou, Jiahao Ge, Hao Xu, and Chi-Wing Fu. Computational design of LEGO® sketch art. *ACM Transactions on Graphics (TOG)*, 42(6), 2023. 3